```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

11. Tutorial - Testing

Kuzey Aydınhan | 06. February 2026

Contents

1. Overview

2. Testing

3. Assertions

4. JUnit

5. Lambda Expressions

Information

- First Abschlussaufgabe release on 16. Feb.
- Next tutorial is the final one
- Any questions/topics you want to go through next week? → Write in ILIAS-Forum

Last Week

Polymorphism

- It is bad practice if you are using:
  → "If class A `doA()`, if class B `doB()`"

```
if (item instanceof A) {
    doA();
} else if (item instanceof B) {
    doB();
}

```

- Instead polymorphie

Last Week

Polymorphism

```java
abstract class Better {
    public abstract void doSomething();
}

class A extends Better {..}

class B extends Better {..}

// With polymorphism
Better item;
item.doSomething();

```

Last Week

equals()

Feature	`==` (Equality Operator)	`.equals()` (Method)
Comparison Type	Compares **references** (memory locations)	Compares **content** (values inside objects)
Works on Primitives?	Yes	No
Works on Objects?	Yes, but only for reference comparison	Yes, for content comparison
Can Be Overridden?	No	Yes
Use Case	Check if two references point to the same object	Check if two objects have identical data

Last Week

`equals()` Dilemma

Symmetry

```
x.equals(y) == y.equals(x)

if (getClass() != o.getClass()) return false;

```

- `parent.equals(child)` (or vice versa) never true

Liskov Substitution Principle

```
parent.equals(child) == true

if (!(o instanceof Parent)) return false;

```

- Breaks when child classes add new fields

Last Week

Dilemma Solution

- Make `equals()` **final** for parent class

```
class Parent {
    @Override
    public final boolean equals(Object obj) {..}
}

```

Last Week

Debugging

- As soon as an error found, the source must be identified and removed
- Print debugging → bad
- Interactive debugging → best

Last Week

Finding Errors

- Static code analyse → meaningful but incomplete
- On end user → bad
- Manually → bad
- Automated → best possible

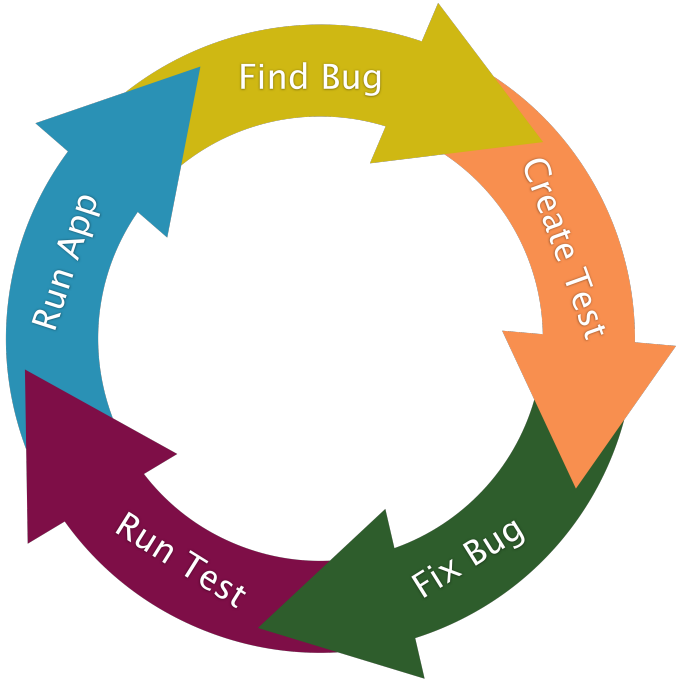
Motivation

- Testing
- Assertions
- JUnit
- Lambda Expressions

Testing

Idea

- Dynamically comparing real value and expected value



Testing
Automated

- Quality assurance and correctness of programs
- Basically finding errors/bugs faster and more efficiently
- Changes can be applied with ease → they get tested instantly
  - Any issues can be solved before committing to the change

Writing Good Tests

- Ideally: Cover all situations where programm can fail
- Procedure:
  1. Choosing the test cases (Classifying possible input values)
  2. Choosing the test data (Representative data for each case)

Choosing Test Cases

Data Based

- Classify by data → each class of inputs (should) result in same output
- Boundaries → negative, `0`, `Integer.MAX_VALUE`, `null`..

Programm Based

- Each line executed (at least) once
- Each branch discovered (at least) once

SWT

- Detailed testing next semester in SWT

Criteria

For Good Tests

- Tests exactly one single property
- As short and concise as possible
- Can be run in isolation (no other dependency to another test)
- Well documented
- Well named → understandable by just looking at name

Test First Development

- Extensive testing is essential

- Write tests as you code
- Better: write tests before coding

Testing

Java

- Assertions
- JUnit

Assertions

Idea

- Meant for logically provable assumptions

- **Precondition:** Restrictions for parameters
- Hold important properties/values of variables inside a method (e.g. loop invariants)
- **Postcondition:** Effects of methods

Takeaway

- Assertions are used for documenting invariants
- Assertions are used for program assurances

Assertions

- **Keyword:** `assert`
- Checks if a condition is held (`boolean`):

```
int result = Calculator.add(9, 10);
assert result == 19;

```

- If it isn't held → throws `AssertionError`
- Can pass an error message:

```
assert result == 19 : "Result does not match expected value: 19";

```

Assertions

- The assertions will not be run in default!
- To run with assertions: `-ea`

  → `java -ea MyClass`

Assertions vs If Statements

Assertions

- Assumptions of the programmer for correct program execution
- Documentation

```
int method(Integer k) {
    assert k != null : "k must not be null";
    return k + 1;
}

```

If Statements

- Prevention of special cases
- Handleable issues
- Will always run

```
int method(Integer k) {
    if (k == null) {
        throw new IllegalArgumentException(..);
    }
    return k + 1;
}

```

Assertions

Assertions should not be used for validating external input!

JUnit

Idea

- A testing framework
- Includes all methods necessary for testing
- After change, can easily run tests

- JUnit guide

JUnit

Testing

3 Phases:

- Construction (Optional)
- Testing
- Cleaning Up (Optional)

JUnit

Test Example

```java
class MyClassTest {
    @Test
    @DisplayName("Addition Test")
    void addsTwoNumbers() {
        // Given
        int first = 1;
        int second = 1;

        // When
        int result = Calculator.add(first, second);

        // Then
        int expectedResult = 2;
        assertEquals(expectedResult, result, "Result should be : $expectedResult");
    }
}
```

Takeaway

- Always test a single thing in a test, don't mix
- Use "given, when, then" for better structure

Construction & Clean Up

- Some requirements must be filled before tests
  → instances set up
  → files loaded
- Sometimes you want to reset the instance to a point before moving on to next test
- `@BeforeEach/@AfterEach` lets a method run before/after **each** test
- `@BeforeClass/@AfterClass` lets a method run before/after **all** tests

Annotations Overview

JUnit 4 Annotation	JUnit 5 Annotation	Description
@Test	@Test	Declares a test.
@Before @After	@BeforeEach @AfterEach	Executed before/after each test.
@BeforeClass @AfterClass	@BeforeAll @AfterAll	Executed once at the beginning / at the end.
@Ignore	@Disabled	Test is not executed.
@DisplayName("Name")	@DisplayName("Name")	Display name of the test.

Methods Overview

Assertion	Description
`fail([message])`	Always produces a failure.
`assertTrue(boolean condition, [message])` `assertFalse(boolean condition, [message])`	Checks a boolean condition.
`assertEquals(expected, actual, [message])` `assertNotEquals(expected, actual, [message])`	Checks the (in)equality of two values.
`assertEquals(expected, actual, tolerance, [message])`	Checks the equality of two floating-point numbers.
`assertNull(object, [message])` `assertNotNull(object, [message])`	Checks whether an object is `null` or not `null`.
`assertSame(expected, actual, [message])` `assertNotSame(expected, actual, [message])`	Checks whether both variables refer to the same or different objects.
`assertThrows(ExceptionClass, Executable)`	Checks whether an exception is thrown.
`assertArrayEquals(expected, actual, [message])`	Checks the equality of all elements in an array.

Test Results

- **Pass:** Test fills its purpose and the tested method acts as expected
- **Fail:** Test fills its purpose and the tested method **doesn't** act as expected
- **Error:** Test doesn't fill its purpose (e.g. an exception in test)

Test Classes

- Structure tests together
- Test only a single class
- Creates and uses references to the classes being tested

Test Classes

```java
public class CustomerTest {
    private static final String PASSWORD = "password";
    private Customer customer;

    @BeforeEach
    void setUp() {
        // ...
        customer = new Customer(name, lastName, username, PASSWORD, personalId);
    }

    @Test
    @DisplayName("Returns true when authenticate")
    void authenticateTest() {
        boolean result = customer.authenticate(PASSWORD);
        assertTrue(result);
    }

    @Test
    @DisplayName("Returns false when authenticate wrong password")
    void authenticateWrongPasswordTest() {
        String invalidPassword = "invalid";
        boolean result = customer.authenticate(invalidPassword);
        assertFalse(result);
    }
}
```

Setting Up JUnit

- Eclipse: https://www.softwaretestinghelp.com/
- IntelliJ: https://www.jetbrains.com/help/idea/

Exercise 1

- There is another exercise in ILIAS
- Download the exercise
- Test the class `Customer`

```java
@Test
@DisplayName("Returns CUSTOMER when get account type")
void getAccountTypeTest() {
    // TODO: Test me
}

@Test
@DisplayName("Returns username when get identifier")
void getIdentifierTest() {
    // TODO: Test me
}

@Test
@DisplayName("Returns empty mailbox when get mailbox")
void getMailboxTest() {
    // TODO: Test me
}

```

Anonym Class

- Sometimes its necessary to create a "temporary" object
- Hence the anonym class

```java
button.addActionListener(
    new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            System.out.println("Button clicked!");
        }
    }
);

```

Lambda Expressions

- Compact expression of an anonym class

```
() -> Expression

(Parameters) -> Expression

(Parameters) -> {
                    Expression;
                    ...
                    return value;  // (if not void)
                }

```

Example

Most Common Usage

```java
List<Integer> list = new ArrayList:>();
// Add some elements ...

// ------ 1. Variation: Anonym Class ------
list.sort(new Comparator<Integer>() {
    @Override
    public int compare(Integer x, Integer y) {
        return y - x;
    }
});

// ------ 2. Variation: Lambda Expression ------
list.sort( (x, y) -> y - x );

```

Example

Most Common Usage

```java
// ------ 3. Variation: Lambda Expression as Variable ------
Comparator<Integer> reverseIntegerComparator = (x, y) -> (y - x);
list.sort(reverseIntegerComparator);

// ------ 4. Varition: Reuse of Other Methods ------
list.sort(MyClass::compareIntegersReversed);


public class MyClass {

    // Parameters should match
    public static int compareIntegersReversed(int x, int y) {
        return y - x;
    }
}

```

Exercise 2

- Write the following using a lambda expression

```java
button.addActionListener(
    new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            System.out.println("Button Clicked");
        }
    }
);

```

Conclusion

- Testing
- Assertions
- JUnit

Last Tutorial

- Any topics you might wanna discuss → write in ILIAS

Exercise 1

```java
@Test
@DisplayName("Returns CUSTOMER when get account type")
void getAccountTypeTest() {
    AccountType result = customer.getAccountType();
    Assertions.assertEquals(AccountType.CUSTOMER, result);
}

@Test
@DisplayName("Returns username when get identifier")
void getIdentifierTest() {
    String result = customer.getIdentifier();
    Assertions.assertEquals(USERNAME, result);
}

@Test
@DisplayName("Returns empty mailbox when get mailbox")
void getMailboxTest() {
    MailBox mailBox = customer.getMailBox();
    boolean hasReceivedMail = mailBox.hasReceivedMail();
    Assertions.assertFalse(hasReceivedMail);
}

```

Exercise 2

```java
button.addActionListener(
    e -> System.out.println("Button Clicked")
);

// Or
button.addActionListener(
    (ActionEvent e) -> System.out.println("Button Clicked")
);

```